REPORT

Reviewer report: Textualize/rich#4137: Fix readline

support (alternative fix)

Run rich-4137, tested 2026-11-08 12:35 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 12:02 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Fix readline support (alternative fix) (Textualize/rich#4137).

• What we observed: 1 test(s) pass on main but fail on the PR; 100 test(s) ran on main, 100 on the PR; 0 new

test(s) pass, 99 unchanged; probe model-probe1 is not_adequate (ran none of the 1 changed line(s) on the main

side); its results do not count either way.

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 4.6 GB (of 15.6 GB).

• Setup check: free disk 93.2 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: rich python setup install instructions https://github.com/Textualize/rich site:github.com/Textualize/rich

• Query: rich python documentation install https://github.com/Textualize/rich site:github.com/Textualize/rich

• Query: rich python install requirements https://pypi.org/project/rich/ https://github.com/Textualize/rich

site:pypi.org

• Source: rich - PyPI — https://pypi.org/project/rich/

• Source: rich-cli - PyPI — https://pypi.org/project/rich-cli/

• Source: rich-pyfiglet - PyPI — https://pypi.org/project/rich-pyfiglet/

• Source: GitHub - Textualize/rich: Rich is a Python library for rich text and beautiful formatting in the terminal. ·

GitHub — https://github.com/textualize/rich

• Source: [BUG] rich fails to install using pip in editable mode · Issue #346 · Textualize/rich · GitHub —

https://github.com/Textualize/rich/issues/346

• Source: GitHub - Textualize/rich-cli: Rich-cli is a command line toolbox for fancy output in the terminal · GitHub

— https://github.com/textualize/rich-cli

• Source: rich/README.ru.md at main · Textualize/rich · GitHub —

https://github.com/textualize/rich/blob/master/README.ru.md

• Source: rich/.readthedocs.yml at main · Textualize/rich · GitHub —

https://github.com/Textualize/rich/blob/main/.readthedocs.yml

• Setup plan: the same packages on main and pr.

• Setup plan: install alabaster==1.0.0 (listed in requirements.txt) on main and pr.

• Setup plan: install Sphinx==8.2.3 (listed in requirements.txt) on main and pr.

• Setup plan: install sphinx-rtd-theme==3.0.2 (listed in requirements.txt) on main and pr.

• Setup plan: install sphinx-copybutton==0.5.2 (listed in requirements.txt) on main and pr.

• Setup plan: install poetry (listed in tox.ini deps) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.9.0

• Setup plan: refused rich-cli: its PyPI page does not link to this GitHub repository

• Installed alabaster 1.0.0 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed Sphinx 8.2.3 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed sphinx-rtd-theme 3.0.2 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed sphinx-copybutton 0.5.2 from requirements.txt (listed in requirements.txt) on main and pr.

• Installed poetry 2.5.1 from tox.ini (listed in tox.ini deps) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C1 | Fix readline support (alternative fix) (PR title) | not testable | claims are read out of the PR description only

by model-written probes (--model-provider, see Probes); the tests below check behavior | pytest-pr-1.log:33 «1

failed, 99 passed in 0.82s»

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | false | 1 test(s) fail on the

PR, e.g. test_console::test_input | pytest-pr-1.log:32 «FAILED tests/test_console.py::test_input - AssertionError:

assert 'foo: \x08'...»

| - | Change not in the PR description (this run) | unmentioned change | 1 test(s) pass on main but fail on the PR,

e.g. test_console::test_input | pytest-pr-1.log:32 «FAILED tests/test_console.py::test_input - AssertionError:

assert 'foo: \x08'...»

Citation check (PASS): 3 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 100 / 100 | | Tests unchanged | 99 | -

| New tests that pass | 0 | | Tests that pass on main but fail on the PR | 1 | test_console::test_input

| Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | test_console::test_input fails with: AssertionError: assert 'foo: \x08' == 'foo:'

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe1.py, frozen

5f805a64b5be at 2026-11-08T07:07:20.862796+00:00): model-written probe of the changed files (no description

given)

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| obs.print.end | - | same on both sides | same on both sides: "hello" | probe-model-probe1-main-1.jsonl:1

«"observation": "obs.print.end"»
probe-model-probe1-pr-1.jsonl:1 «"observation": "obs.print.end"»

| obs.print.multiple | - | same on both sides | same on both sides: "hello world\n" |

probe-model-probe1-main-1.jsonl:2 «"observation": "obs.print.multiple"»
probe-model-probe1-pr-1.jsonl:2

«"observation": "obs.print.multiple"»

| obs.print.sep | - | same on both sides | same on both sides: "hello, world\n" | probe-model-probe1-main-1.jsonl:3

«"observation": "obs.print.sep"»
probe-model-probe1-pr-1.jsonl:3 «"observation": "obs.print.sep"»

| obs.print.simple | - | same on both sides | same on both sides: "hello\n" | probe-model-probe1-main-1.jsonl:4

«"observation": "obs.print.simple"»
probe-model-probe1-pr-1.jsonl:4 «"observation": "obs.print.simple"»

| obs.print.style | - | same on both sides | same on both sides: "hello\n" | probe-model-probe1-main-1.jsonl:5

«"observation": "obs.print.style"»
probe-model-probe1-pr-1.jsonl:5 «"observation": "obs.print.style"»

Citation check (PASS): 10 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (3.4 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| rich/console.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 45f7925894a6 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: pytest exit 1; no identical pre-existing failures.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 3 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 10 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | 1 test(s) pass on main but fail on the PR; 100 test(s) ran on main, 100 on

the PR; 0 new test(s) pass, 99 unchanged; probe model-probe1 is not_adequate (ran none of the 1 changed

line(s) on the main side); its results do not count either way | This is about the PR, not the run: read the failing

tests and their log lines, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | pytest exit 1; no identical pre-existing failures

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 3 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 10 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-08/rich-4137 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/Textualize/rich/pull/4137

Evidence links

• Pull request Textualize/rich#4137

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-probe1-main-1.jsonl

• probe-model-probe1-pr-1.jsonl

• main run 1 result: pytest-main-1.log:8 «100 passed in 0.84s»

• pr run 1 result: pytest-pr-1.log:33 «1 failed, 99 passed in 0.82s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 12:02 IST